

通信販売システム ソフトウェア詳細設計書

SWSS-001

承認	作成
佐藤	鈴木
2025/7/8	2025/7/8

2025年7月8日 IPUT-OK 情報工学科

通信販売システム ソフトウェア詳細設計書

改訂履歴

日付	バージョン	改訂内容	担当
2025/7/8	Ver.0.1	初版作成	鈴木

通信販売システム ソフトウェア詳細設計書

目次

- 1. 概要
- 2. プログラムユニット機能と構成設計 2.1 プログラムユニット一覧表 2.2 プログラムユニット構成図
- 3. プログラムユニット設計 3.1 プログラムユニット詳細処理 3.2 プログラムユニット状態遷移 3.3 リソース定義 3.4 システム初期化处理 3.5 共通定義
- 4. プログラムユニット・インタフェース設計 4.1 プログラムユニット間のインタフェース 4.2 インタフェース詳細

通信販売システム ソフトウェア詳細設計書

1. 概要

- **本書の目的** 本書は、通信販売システムのプログラムレベルの仕様(詳細な振る舞いや論理構造など)を記述する。

- **本書の位置づけ** 本書は、通信販売システムの3号開発ドキュメントであり、後続のソースコーディングのインプット資料として使われる。
- **対象ユーザ** ソフトウェアの実装担当者(プログラマ)、ソフトウェア単体テスト設計者。
- **記載範囲、記載内容など** 通信販売システムの実装すべき全てのプログラムを明確にし、コーディング可能なレベルに記述する。
- **参照しているドキュメントなど** 「通信販売システム ソフトウェア方式設計書」
- **定義(用語、略語など)**

通信販売システム ソフトウェア詳細設計書

2. プログラムユニット機能と構成設計

2.1 プログラムユニット一覧表

機能カ テゴリ	プログラム名	機能概要
注文管 理機能	check_stock	注文内容に基づき、該当商品の在庫を確認する。
	register_order	購入者情報、支払い方法を含めた注文内容をシステムに入力し、注文番号と注文日を自動生成して登録する。
	manage_payment	支払い方法（銀行振込、コンビニ決済、代金引換）に応じた処理を行う。
	select_product	商品リストから希望する商品と数量を選択する。
	input_customer_info	氏名、住所、連絡先を入力する。
	process_payment	クレジットカード情報を入力し、決済を完了する。
	confirm_order	入力された情報に基づき注文を確定し、注文データを生成する。
商品管 理機能	update_stock_quantity	注文の発送状態が「発送済」となった際に、在庫数量を自動的に更新する。
	query_stock_quantity	在庫数量（実在庫と有効在庫）を照会する。
	search_product	商品番号または商品名で商品を検索する。

機能力 テゴリ	プログラム名	機能概要
顧客管 理機能	manage_customer_info	購入者情報（氏名、住所、連絡先など）を登録、更新、照会する。
会計管 理機能	confirm_payment_status	注文情報を検索し、入金確認後に支払い状態を「支払済」に更新する。
発送管 理機能	view_unshipped_orders	未発送の注文データを閲覧する。
	create_invoice	納品書や請求書を作成する。
	pick_products	倉庫から商品をピッキングする。
	update_shipping_status	発送後、注文の発送状態を「発送済」に変更する。
データ 管理機 能	manage_order_data	注文データの登録、更新、検索、削除を行う。
	manage_product_stock_data	商品在庫データの登録、更新、検索を行う。

2.2 プログラムユニット構成図

```

classDiagram
    class Customer {
        -customerID
        -name
        -address
        -contactInfo
        +register()
        +updateInfo()
        +search()
    }
    class Order {
        -orderID
        -orderDate
        -customerID
        -paymentMethod
        -shippingStatus
        +createOrder()
        +updateStatus()
        +deleteOrder()
    }
    class Product {
        -productID
        -productName
  
```

```
-price
-stockQuantity1
-stockQuantity2
+updateStock()
+queryStock()
+searchProduct()
}
class OrderManagement {
    +receiveOrder()
    +registerOrder()
    +confirmOrder()
}
class ProductManagement {
    +manageStock()
    +searchProduct()
}
class CustomerManagement {
    +manageCustomerInfo()
}
class AccountingManagement {
    +confirmPayment()
}
class ShippingManagement {
    +processShipment()
    +createDocuments()
    +updateShippingStatus()
}

Customer "1" -- "*" Order
Order "1" -- "*" Product
OrderManagement -- Order
ProductManagement -- Product
CustomerManagement -- Customer
AccountingManagement -- Order
ShippingManagement -- Order
```

通信販売システム ソフトウェア詳細設計書

3. プログラムユニット設計

3.1 プログラムユニット詳細処理

				制 約 事 項
プログラム名	引数	戻り値	処理内容	

プログラム名	引数	戻り値	処理内容	制約事項
register_order	注文情報（購入者情報、商品、支払い方法）	注文結果 (0:成功, 1:失敗)	注文内容をシステムに登録し、注文番号を発行する。	
update_stock_quantity	商品番号, 数量	更新結果 (0:成功, 1:失敗)	発送済商品の在庫数量を更新する。	
search_product	商品番号 or 商品名	商品情報リスト	条件に一致する商品を検索し、在庫情報と共に返す。	
confirm_payment_status	注文番号	更新結果 (0:成功, 1:失敗)	入金を確認し、注文の支払い状態を「支払済」に更新する。	
update_shipping_status	注文番号	更新結果 (0:成功, 1:失敗)	商品発送後、発送状態を「発送済」に更新する。	

3.2 プログラムユニット状態遷移

注文管理状態遷移図

```
stateDiagram-v2
    [*] --> 注文受付
    注文受付 --> 在庫確認
    在庫確認 --> 支払い処理 : 在庫あり
    在庫確認 --> [*] : 在庫なし
    支払い処理 --> 発送処理 : 支払い完了
    支払い処理 --> 注文キャンセル : 支払い失敗
    発送処理 --> 在庫更新
    在庫更新 --> 完了
    完了 --> [*]
```

3.3 リソース定義

リソース名	用途	構造	値とその意味
-------	----	----	--------

リソース名	用途	構造	値とその意味
顧客マスタ	顧客情報を管理する	顧客ID, 氏名, 住所, 連絡先	
商品マスタ	商品情報を管理する	商品番号, 商品名, 単価	
在庫テーブル	商品の在庫を管理する	商品番号, 在庫数量1, 在庫数量2	在庫数量1: 実在庫, 在庫数量2: 有効在庫
注文データ	注文情報を管理する	注文番号, 注文日, 顧客ID, 支払い方法, 支払い状態, 発送状態	

通信販売システム ソフトウェア詳細設計書

3.4 システム初期化处理

- データベースサーバとの接続を確立する。
- 各マスタデータ、テーブルへのアクセス準備を行う。
- Webサーバの初期設定を読み込み、リクエスト待機状態にする。

3.5 共通定義

- エラーコードリスト: E001=DB接続エラー, E002=在庫不足, E003=決済失敗
- 状態コード: `payment_status` (0:未払い, 1:支払済), `shipping_status` (0:未発送, 1:発送準備中, 2:発送済)

通信販売システム ソフトウェア詳細設計書

4. プログラムユニット・インタフェース設計

4.1 プログラムユニット間のインタフェース

- 各機能管理ユニット内部のプログラムユニット間は、直接call。
- Webクライアントとサーバ間は、HTTP/HTTPSプロトコルを用いたAPIでやり取りを行う。

4.2 インタフェース詳細

インターフェース	呼び出し方法	結果	データフォーマット
----------	--------	----	-----------

インターフェース	呼び出し方法	結果	データフォーマット
ネット注文受付 (confirm_order)	POST /api/orders	注文結果 (JSON)	Request Body (JSON): { customerInfo: {...}, items: [...], payment: {...} } Response Body (JSON): { orderId: "...", status: "success" }
商品検索 (search_product)	GET /api/products?q=...	商品リスト (JSON)	Response Body (JSON): [{ productId: "...", name: "...", price: ..., stock: ... }]